



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 10: Consola Logística

"ExpresoFast" - Migración a Frontend SPA con Angular Standalone y Backend RESTful por Capas

Metadatos

- **Tiempo Estimado:** 6 horas (trabajo autónomo individual)

Herramientas Requeridas:

- Java 21 (o versión local instalada) con Maven.
- Spring Boot 3.x (Spring Data JPA, Hibernate, OpenAPI/Swagger).
- Node.js 18+ y Angular CLI (@angular/cli).
- H2 Database o SQL Server Developer Edition.

- Navegador Web moderno (Google Chrome / Firefox) con DevTools.
- Editor de código (Visual Studio Code / IntelliJ IDEA).
- Git y Cuenta activa de GitHub.

Metas de Aprendizaje:

Estructurar una solución de software empresarial desacoplada dividida en dos repositorios/carpetas independientes (expresofast-backend y expresofast-frontend).

Diseñar y construir una arquitectura backend RESTful por capas en Spring Boot (Dominio, Repositorio, DTOs, Servicio y Controlador) con soporte explícito de políticas CORS para Angular (<http://localhost:4200>).

Inicializar y estructurar una aplicación Single Page Application (SPA) cliente en **Angular Standalone (Angular 19/18)** utilizando Angular CLI (`ng new expresofast-frontend --standalone`).

Configurar la inyección de dependencias de `HttpClient` (`provideHttpClient(withFetch())`) y variables de entorno (`environment.ts`) en la aplicación Angular.

Desarrollar modelos de datos TypeScript, servicios HTTP reactivos (`HttpClient`) y componentes Standalone enrutables (`app.routes.ts`) con plantillas HTML, estilos CSS y data binding (`[(ngModel)]`).

Demostrar autonomía en la resolución de problemas de integración cliente-servidor, depuración de rutas y manejo de errores HTTP.

Introducción y Caso de Estudio (Sexta Parte)

En los laboratorios previos (**Laboratorio 8 y 9**), su equipo desarrolló y optimizó los endpoints RESTful de la consola logística **ExpresoFast** e integró una interfaz cliente en Vanilla JavaScript (Fetch API) con controles de paginación relacional y procedimientos almacenados.

Siguiendo el plan de modernización de la arquitectura tecnológica de **ExpresoFast**, la Gerencia de Sistemas ha ordenado la **migración total del cliente web a una Single Page Application (SPA) basada en el framework Angular Standalone**.

Para esta **Sexta Parte**, usted debe desarrollar de forma **autónoma e individual** el frontend en Angular Standalone (expresofast-frontend) y conectarlo con la API RESTful de Spring Boot (expresofast-backend). Aplicando los conocimientos adquiridos en la **Práctica Guiada 10** y **Práctica Guiada 10.a**, usted debe demostrar su capacidad para estructurar componentes, servicios, modelos e interfaces de usuario reactivas sin depender de guías paso a paso.

Requerimientos Técnicos de la Solución

1. Estructura del Proyecto en Repositorio GitHub

Su repositorio de entregable debe organizarse estrictamente en dos directorios principales:

```
expresofast-project/
├── expresofast-backend/      <-- Proyecto Spring Boot (Java 21)
│   ├── src/main/java/com/expresofast/
│   │   ├── model/Envio.java
│   │   ├── repository/EnvioRepository.java
│   │   ├── dto/
│   │   │   ├── EnvioDTO.java
│   │   │   ├── CrearEnvioDTO.java
│   │   │   └── service/
│   │   │       ├── EnvioService.java
│   │   │       └── EnvioServiceImpl.java
│   │   └── controller/EnvioController.java
│   └── src/main/resources/application.properties
└── expresofast-frontend/    <-- Proyecto Angular Standalone
    ├── src/environments/
    │   └── environment.ts
    ├── src/app/
    │   ├── models/envio.model.ts
    │   ├── services/envio.service.ts
    │   ├── components/
    │   │   └── envio-list/
    │   │       ├── envio-list.component.ts
    │   │       └── envio-list.component.html
```

```

■ ■ ■ ■ ■ envio-list.component.css
■ ■ ■ ■ ■ envio-form/
■ ■ ■ ■ ■ envio-form.component.ts
■ ■ ■ ■ ■ envio-form.component.html
■ ■ ■ ■ ■ envio-form.component.css
■ ■ ■ ■ ■ envio-tracking/
■ ■ ■ ■ ■ envio-tracking.component.ts
■ ■ ■ ■ ■ envio-tracking.component.html
■ ■ ■ ■ ■ envio-tracking.component.css
■ ■ ■ ■ ■ app.config.ts
■ ■ ■ ■ ■ app.routes.ts
■ ■ ■ ■ ■ angular.json
■ ■ ■ ■ ■ package.json

```

2. Especificación de la Capa Backend (expresofast-backend)

- **[] Entidad JPA (Envio.java):**

Atributos: `id` (Long), `codigoRastreo` (String, único), `destinatario` (String), `direccionDestino` (String), `montoFlete` (Double), `estado` (String: PENDIENTE, EN_TRANSITO, ENTREGADO, CANCELADO), `fechaCreacion` (LocalDateTime).

- **[] DTOs (EnvioDTO.java y CrearEnvioDTO.java):**

Estructura DTO limpia para transferir datos hacia y desde la API REST.

- **[] Repositorio (EnvioRepository.java):**

Consultas personalizadas JPQL o Spring Data JPA (búsqueda por código de rastreo y filtrado por estado).

- **[] Servicio de Negocio (EnvioService):**

Métodos para obtener todos los envíos, buscar por código de rastreo, registrar un nuevo envío (generando un código de rastreo único ej: `EXP-2026-XXXX`) y actualizar el estado de entrega.

- **[] Controlador RESTful (EnvioController.java):**

- RUTA BASE: `/api/v1/envios`

- `GET /api/v1/envios`: Retorna la lista de todos los envíos.
- `GET /api/v1/envios/rastreo/{codigo}`: Retorna el detalle del envío según el código de rastreo.
- `POST /api/v1/envios`: Registra un nuevo envío desde el payload `CrearEnvioDTO`.
- `PATCH /api/v1/envios/{id}/estado`: Actualiza el estado de entrega del envío.
- **CORS**: Incluya `@CrossOrigin(origins = "http://localhost:4200")` en el controlador para autorizar peticiones del cliente Angular.

3. Especificación de la Capa Frontend (`expresofast - frontend`)

- **[] Creación del Proyecto Angular CLI:**

Genere el proyecto ejecutando: `ng new expresofast-frontend --standalone (CSS format, SSR No)`.

[] Configuración Global (`app.config.ts` y `environment.ts`):

- En `src/environments/environment.ts`, configure `API_URL`:
`'http://localhost:8080/api/v1/'`.

En `src/app/app.config.ts`, incluya `provideHttpClient(withFetch())`.

[] Modelos TypeScript (`src/app/models/envio.model.ts`):

Declare las interfaces/clases `Envio` y `CrearEnvioPayload`.

[] Servicio Angular HTTP (`src/app/services/envio.service.ts`):

- Inyecte `HttpClient` con `inject(HttpClient)`.
- Implemente métodos HTTP reactivos: `obtenerEnvios()`, `obtenerPorRastreo(codigo)`, `crearEnvio(payload)`, `actualizarEstado(id, nuevoEstado)`.

[] Componente Dashboard de Envíos (`EnvioListComponent`):

- **Ruta:** `/envios`

- Vista en tabla/grid de guías de envío con código de rastreo, destinatario, dirección, flete y estado.
- Insignia de color según el estado (`PENDIENTE`: amarillo, `EN_TRANSITO`: azul, `ENTREGADO`: verde, `CANCELADO`: rojo).

Selector desplegable para actualizar el estado del envío directamente desde la tabla con llamada a la API REST.

[] **Componente Formulario de Registro (`EnvioFormComponent`):**

- **Ruta:** `/nuevo-envio`
- Formulario con data binding bidireccional [`(ngModel)`] para registrar destinatario, dirección y monto de flete.
- Validación de campos obligatorios e integración con `EnvioService.crearEnvio()`.

Redirección o mensaje de éxito al completar el registro.

[] **Componente Rastrear Guía (`EnvioTrackingComponent`):**

- **Ruta:** `/rastreo`
- Campo de búsqueda por código de rastreo (ej: EXP-2026-1001).

Despliegue de ficha detallada del paquete con barra de progreso de estado de entrega.

[] **Tabla de Rutas y Navegación (`app.routes.ts` y `app.component.html`):**

- Configure la tabla de rutas en `app.routes.ts` con redirección por defecto a `/envios`.
- En `app.component.html`, implemente una barra de navegación superior accesible con enlaces `routerLink` hacia las 3 vistas.

Guía de Depuración de Errores Comunes en Angular

Error 1: Bloqueo de Peticiones por CORS (`Access-Control-Allow-Origin`)

- **Causa:** El backend Spring Boot rechaza las peticiones HTTP realizadas desde el origen `http://localhost:4200`.
 - **Verificación:** Abra F12 DevTools pestaña **Console**. Si visualiza un error de política CORS, asegúrese de agregar `@CrossOrigin(origins = "http://localhost:4200")` sobre la clase `@RestController` en Java.
-

Error 2: `NullInjectorError: No provider for HttpClient!`

- **Causa:** Olvidó registrar el cliente HTTP en el contenedor de dependencias globales de Angular.
 - **Verificación:** Compruebe que `provideHttpClient(withFetch())` esté presente dentro del arreglo `providers` en `src/app/app.config.ts`.
-

Error 3: El formulario no actualiza las propiedades (`Can't bind to 'ngModel'`)

- **Causa:** La directiva `[(ngModel)]` requiere importar `FormsModule`.
 - **Verificación:** En la clase del componente `Standalone (.component.ts)`, agregue `FormsModule` al arreglo `imports: [CommonModule, FormsModule]`.
-

Entregables y Modalidad de Entrega

- **Modalidad:** Trabajo Autónomo Individual.

Entrega en GitHub:

- Suba el proyecto estructurado en las carpetas `/expresofast-backend` y `/expresofast-frontend`.
- Incluya un archivo `README.md` detallando las instrucciones de compilación y ejecución (`mvnw spring-boot:run` y `ng serve`).

Entrega en Mediación Virtual:

- Suba un documento PDF comprimido que contenga:
 1. Enlace al repositorio público en GitHub.
 2. Capturas de pantalla de la barra de navegación y las 3 vistas en Angular corriendo en el navegador web (`/envios`, `/nuevo-envio`, `/rastreo`).
 3. Captura del proceso de actualización de estado de un envío y registro de una nueva guía con persistencia en la base de datos backend.

Rúbrica de Evaluación (Total: 100 Puntos)

Criterio de Evaluación	Puntuación	Descripción
Backend RESTful & Capas Spring Boot	25 pts	Estructura limpia en capas (Entidades, Repositorios, DTOs, Servicios, Controladores), endpoints de consulta, registro y actualización con soporte CORS.
Inicialización y Arquitectura Angular	20 pts	Espacio de trabajo Angular Standalone correctamente estructurado (models, services, components, environments), proveedor HTTP y tabla de rutas.

Servicios HTTP Reactivos (<code>HttpClient</code>)	20 pts	Implementación de <code>EnvioService</code> consumiendo la API REST backend asíncronamente (GET, POST, PATCH).
Componentes e Interfaz SPA	25 pts	Desarrollo de los 3 componentes (<code>EnvioListComponent</code> , <code>EnvioFormComponent</code> , <code>EnvioTrackingComponent</code>) con data binding, insignias CSS de estado y navegación sin recarga.
Documentación y Repositorio GitHub	10 pts	Estructura de repositorio limpia en dos carpetas, historial de commits individuales significativos y archivo <code>README.md</code> explicativo.